

University of Lincoln Assessment Framework

Assessment 2 Briefing 2024-2025

1. Module code & title	CMP1902M Programming Fundamentals
2. Assessed learning outcomes	On successful completion of this component a student will have demonstrated competence in the following areas: • [LO1] Identify, select, and apply appropriate data structures and operators in common programming solutions • [LO2] Implement control flow with decisions and loops using industry-informed programming practices and professional standards • [LO3] Using appropriate knowledge of programming concepts, construct code segments and functions to perform input and output operations with error handling
3. Assessment title	Assessment 2
4. Contribution to final module mark (%)	80%
5. Description of assessment task	This is Assessment 2 and is an individual assignment that consists of two tasks. Task 1 contributes 20% of the assessment mark and Task 2 contributes 80%. You can use the provided code as a template to help you begin the implementation of the tasks. Please see the Criterion Reference Grid for details of how tasks will be graded. Task 1 (20%): Number Statistics In this task, you are asked to write a Python3 script (task1.py) that: • Prompts the user to input only positive integer numbers at the terminal (numbers with decimals, negative numbers or characters will not be accepted) • Assign those numbers to a list and removes any duplicate numbers and informs the user of the removed duplicates • Counts the number of unique numbers in the list. • Calculates their product, range and variance • Identifies the even and odd numbers from the list and stores them in separate lists. If the list of even numbers is empty, print

a message saying "No even numbers were provided." If the list of odd numbers is empty, print a message saying "No odd numbers were provided."

- Prints all the above information at the terminal

Important: The script should start and end without problems. There should be no 'crashing' or 'unhandled exceptions' due to erroneous input. The use of any built-in method that performs the above operations is not allowed. Importing of modules is also not allowed. The code must be commented, including header comments and in-line comments. Comments in the code should follow the PEP8 style. Consideration must be given to the naming convention of variables and function names.

Task 2 (80%): Variant of the Wordle game

Background: Wordle is a word-based game released in 2013 and gained significant popularity since then. In the original version, a user must guess a 5-letter word with six tries. You can play the game [here](#). *For this task you are expected to develop a variant of the game as described below*

Task: For this task you will have to implement a python3 script (task2.py) with multiple functions and appropriate comments at the code following the PEP 8, that enables a player to play a variant of the Wordle game using only the terminal (graphical user interface is not required) as follows:

Basic Features:

- The script will randomly select one 5 letter word from the given dictionary (*dictionary.txt*). This random word is the answer that the user must find within 6 turns.
- During each turn the user provides a word as a guess. This must be a real word i.e. a word that exists within the given dictionary. If the word does not exist, the user gets a warning and losses a turn.
- After every guess, the code should provide a clue by assigning the symbols: *, + and _ for each letter of the guess. The symbol * indicates that the letter is correct and in the correct position, the symbol + means that the corresponding letter is in the answer but not in the right position, while the symbol _ indicates the corresponding letter is not in the answer at all. The above symbols should be separated by space when printed at the terminal. For example: if the user guess is the word: *audio*, and the provided clue from the program is _ * _ + _, it means that the letter u is part of the randomly selected word and at the correct position, the letter i is also part of the randomly selected

word but it is not at the correct position. The other letters (a,d,o) are not part of the random word. In this case the correct answer could be the word: judge

- Multiple instances of the same letter in a guess, such as the "o"s in "robot", will be assigned a * or + only if the letter also appears multiple times in the answer; otherwise, all the repeated letters will be assigned with _.
- The player should have the option to give up at any-time by ending the game using an appropriate input. In this case, the randomly selected word should be revealed.
- If the player fails to find the answer within 6 turn or gives up, they lose the game. If the correct answer is found, they win the game. In both cases, an appropriate message should be printed.

Desired Features:

- The code should ask the user at the beginning of the game, to provide a name or alias and measure the time in seconds that they needed to solve the puzzle. That info should be stored in a winners.txt file which has to be updated only after a win. The user may have the option to see the past winners and their time (contents of the winners.txt) before the beginning of a puzzle.
- After each guess from the user, the programme should provide, in addition to the clue, a list with all the letters that the user has used but are not part of the correct answer.
- The player is given 30 seconds to make a guess. If they provide a word after the 30 seconds, their turn is lost
- Any erroneous input from the players should be handled appropriately and not result to crashing or unhandled exceptions.

Advanced Features:

- The player can select to play with a four, five or six letter word and they also have the option of a hard mode for each. The hard mode requires players to include letters marked as * and + in subsequent guesses.
- The player has the option to use a hint only once. This hint will provide a letter that is part of the answer but will not provide its location. The player loses one turn when using the hint
- The player can ask for help with the vocabulary. This will provide all the words in the dictionary that are tentative solutions and satisfy the clues that have been provided so far during the game.

Important: You can import the following four modules: time, sys, random, NumPy. Any other modules except user-defined ones are prohibited for use. The program should run 'cleanly'. It should start and

	end without problems. There should be no 'crashing' or 'unhandled exceptions' due to erroneous input. The code must be commented, including header comments and in-line comments. Comments in the code should follow the PEP8 style. Consideration must be given to the naming convention of variables and function names.
6. Assessment submission instructions	Please zip up your code files (.py files) as a compressed ZIP file (no RAR or any other file formats will be accepted. Please use the standard 'Windows Compressed File' zip format or other platform standard zip formats) and should be submitted through Blackboard in the ' <u>CMP1902M Assessment Item 2 Supporting Documentation Upload</u> ' section of the Assessments folder. You can make multiple submissions, but only the last one will be marked. The file should be named using the format of 'Initials_Surname.zip'. This assessment is graded using a criterion reference grid. You should be clear in your understanding of the grading principles; if you are not, please seek the advice of the module staff. The deadline for submission of this work is included in the School Submission dates on Blackboard.
7. Date for return of mark and feedback	Please see the Hand In Dates.xls spreadsheet. <i>Note: all marks awarded are provisional until confirmed by the Board of Examiners.</i>
8. Feedback format	Written feedback will be given to you through Blackboard.
9. Use of Artificial Intelligence (AI) in this assessment	<u>The use of AI (Artificial Intelligence) tools to generate all or part of your assessment submission is not permitted</u>
10. Marking criteria for assessment	A Criterion Reference Grid (CRG) is used to evaluate your learning against a set of pre-defined criteria. See CMP1902M Programming Fundamentals, Assessment Item 2, CRG 2425
11. Additional information (support, advice, tips etc)	If you are unsure about any aspect of this assessment document, please seek advice from a member of the delivery team. Your work will be checked for any dishonesty and plagiarism. Please make sure you have a clear understanding of the grading principles for this component as detailed in the accompanying Criterion Reference Grid.
12. Important Information on Dishonesty, Plagiarism and AI Tools	University of Lincoln Regulations define plagiarism as ' <i>the passing off of another person's thoughts, ideas, writings or images as one's own...</i> '. Examples of plagiarism include the unacknowledged use of another person's material whether in original or summary form. Plagiarism also includes the copying of another student's work'. Plagiarism is a serious offence and is treated by the University as a form of academic dishonesty.

	For more information on examples of Academic Offences, please see the Academic Offence Guidance. Please note, if you use AI tools in the production of assessment work where it is not permitted, then it will be classed as an academic offence and treated by the University as a form of academic dishonesty. Students are directed to the University Regulations for details of the procedures and penalties involved. For further information, see www.plagiarism.org
--	--